

Backend Knowledge

Key Concepts Explained

1. Controller

- **What:** A controller is a class that **receives HTTP requests** (like GET, POST, PUT, DELETE) from clients (e.g., browsers, mobile apps).
 - **Role:** It acts like a traffic cop, **routing requests to the right method** and returning responses.
 - **Example:** Your `GiftCardController` listens for requests like `GET /gift-cards/{id}` and calls the service layer to get the data.
 - `ResponseEntity` is a class that represents the entire HTTP response(found, not found, OK)
 - `Controller` injects `Service` as that actually has the business logic.
-

2. Service

- **What:** A service contains **business logic** — the “rules” of your application.
 - **Role:** It handles **how data should be processed, validated, or manipulated** before sending it back or saving it.
 - **Example:** `GiftCardService` retrieves gift card info from the database via a DAO, possibly applies rules, and returns it to the controller.
-

3. DAO (Data Access Object)

- **What:** A DAO is responsible for **talking to the database**.
 - **Role:** It executes queries or commands to **fetch, update, insert, or delete data**.
 - **Example:** `GiftCardDao` runs SQL queries to get gift card info from the database.
-

4. Resource (like `GiftCardResource`)

- **What:** A resource organizes and shapes **how data is sent in HTTP responses**.

- **Role:** It structures the data neatly and can add **links to related data** to help clients navigate the API.
- **Example:** `GiftCardResource` wraps gift card data and adds a **self link** so clients know the URL to get that gift card again.

Resource.java (or more accurately, its modern equivalent EntityModel.java and related classes like CollectionModel.java and RepresentationModel.java) and HATEOAS are fundamentally used together in Spring to embed links within RESTful API responses.

5. HATEOAS (Hypermedia As The Engine Of Application State)

- **What:** A fancy name meaning **your API responses include links to related endpoints**.
- **Role:** Helps clients **discover actions they can take next**, like “here’s the gift card, and here’s a link to update it or see transactions.”
- **Why useful:** It makes APIs more flexible and self-describing.
- **Example:** `.withSelfRel()` adds a “self” link to the resource so clients can easily find their way.

Think of it like browsing a website: you don’t memorize all the URLs of a website beforehand. Instead, you click on the links provided on the pages to navigate to different sections or perform various actions. HATEOAS applies this same principle to APIs, allowing clients to discover and interact with the API dynamically without needing prior knowledge of its structure

6. Serialization

- **What:** Serialization is the process of **converting your Java objects into JSON (or XML) format** that can be sent over HTTP.
 - **Role:** Makes it possible to send complex data structures as plain text that clients can understand.
 - **Jackson:** A popular Java library that handles serialization/deserialization between Java objects and JSON.
 - **Example:** `GiftCardResource` configures what fields are included/excluded in JSON using Jackson mixins.
-

7. Mixin

- **What:** A Jackson feature to **customize what properties get serialized/deserialized without modifying the original class**.
 - **Role:** Helps exclude sensitive or unnecessary fields from JSON responses.
 - **Example:** You ignore fields like `giftCardLoadTranList` and `transactionList` in JSON using a mixin.
-

8. Testing

- **Unit Tests (JUnit, Mockito):** Test individual pieces like services or controllers to make sure they behave correctly in isolation.
 - **Integration Tests (@WebMvcTest):** Test how the web layer (controllers) works end-to-end with mock dependencies.
 - **Postman:** Tool to manually test API endpoints by sending HTTP requests and verifying responses.
-

How These Pieces Work Together in Your Task

1. **You create** `GiftCardController` with a GET endpoint `/gift-cards/{giftCardId}` that receives requests.
2. This controller calls `GiftCardService`, which contains business logic to **get the gift card data**.
3. `GiftCardService` uses the `GiftCardDao` to **retrieve data from the database**.
4. The data is wrapped into a `GiftCardResource`, which shapes the response and adds useful **HATEOAS links**.
5. **Jackson serializes** this resource into JSON, hiding unnecessary details using a **Mixin**.
6. You write **unit tests** for your service and controller to make sure everything works as expected, including error cases.
7. Finally, you use **Postman** for **end-to-end testing** by calling the actual live API and verifying the response.

Summary

Term	Simple Meaning	Role in Your Project
Controller	Receives HTTP requests, sends responses	Entry point for <code>/gift-cards/{id}</code>
Service	Business logic and data processing	Gets gift card info from DAO
DAO	Database access object	Fetches gift card from DB
Resource	Structures API response data	Formats gift card + adds HATEOAS links
HATEOAS	Adding useful links in API responses	<code>.withSelfRel()</code> for self link
Serialization	Convert Java objects to JSON for HTTP responses	Jackson converts <code>GiftCardResource</code>
Mixin	Customize JSON output without changing classes	Hide fields like transactions
Testing	Verify correctness with unit and integration tests	JUnit, Mockito, Postman

	<code>@PathVariable</code>	<code>@RequestBody</code>
Purpose	Extracts value from the URL path. Used for identifying specific resources.	Binds the request body (usually JSON) to a Java object
Location in HTTP Request	URL (e.g., <code>/users/{id}</code>)	HTTP body (e.g., POST/PUT JSON payload)
Used With HTTP Methods	Commonly with <code>GET</code> , <code>DELETE</code> , <code>PUT</code>	Commonly with <code>POST</code> , <code>PUT</code> , <code>PATCH</code>
Data Format	Usually simple types (e.g., <code>String</code> , <code>Long</code> , <code>UUID</code>)	Typically complex types (custom DTOs or domain models)
Annotation Location	On method parameter	On method parameter
Example URL	<code>/users/5</code>	<code>/users</code>
Example Usage in Code	<pre>@GetMapping("/users/{id}") public User get(@PathVariable Long id)</pre>	<pre>@PostMapping("/users") public User save(@RequestBody User user)</pre>

	@PathVariable	@RequestBody
Validation Possible?	Yes, but mostly for type safety	Yes, supports full @Valid or @Validated bean validation
Serialization/Deserialization	Not required	Automatically deserialized from JSON to Java object
Multiple Usage in One Method	Yes, can use multiple path variables	Only one @RequestBody allowed per method

In essence, `@PathVariable` is for values that are part of the URL's structure and identify specific resources, whereas `@RequestBody` is for transferring entire data objects or complex data structures within the request's payload. `{{base_url}}/model-api/check-order-items/92038194`

```
@GetMapping("/check-order-items/{checkOrderId}")
public ResponseEntity<CheckOrderItemResource>
getCheckOrderItem(@PathVariable("checkOrderId") Long checkOrderId) {
    return ResponseEntity.ok().cacheControl(CacheDefaults.ONE_MINUTE).body(new
    CheckOrderItemResource(orderService.getCheckOrderItemById(checkOrderId)));
}
```

Here, **@PathVariable** puts `checkOrderId` into URL : `/check-order-items/{checkOrderId}`

What is a DTO?

DTO stands for **Data Transfer Object**. It's a simple Java class used to **carry data** between processes — typically between a client (like frontend or API consumer) and the server (backend code like your controller/service).

In Your Case: `CheckOrderItemDTO`

It's the **input format** your PATCH endpoint accepts.

So, in your controller:

```
@PatchMapping("/check-order-items/{checkOrderId}")
public CheckOrderItemResource updateCheckOrderItem(@RequestBody
    CheckOrderItemResource.CheckOrderItemDTO dto, @PathVariable Long
```

```
checkOrderItemId) {  
    ...  
}
```

You're saying:

➡ "Take the data the client sends in the body of the request and map it to `CheckOrderItemDTO`."

--

Example: What might be in `CheckOrderItemDTO` ?

Here's a simplified version of what it could look like:

```
public static class CheckOrderItemDTO {  
    private Long itemId;  
    private Long checkOrderId;  
    private BigDecimal quantity;  
    private BigDecimal price;  
    private String notes;  
    // Getters and setters  
}
```

 Example JSON Input (client PATCH request body):

```
{  
  "itemId": 123,  
  "quantity": 2,  
  "notes": "Changed to medium size"  
}
```

This JSON gets deserialized into `CheckOrderItemDTO` and passed into your controller method.

Why not just use `CheckOrderItem` ?

Because DTOs:

- Let you control **what fields are allowed to change**.
- Allow **validation** and transformation.

- Avoid exposing internal database models directly to the client.

GiftCardControllerTest

```
private final GiftCard giftCard = giftCardMock(1L);
```

This means:

You're creating a mock GiftCard object by calling a utility method `giftCardMock(...)`.

You're passing the ID 1L into that method – meaning you want to create a GiftCard with an ID of 1.

When you write:

```
when(giftCardService.getGiftCard(anyLong())).thenReturn(null);
```

You're intentionally forcing the service to return null, so that your controller behaves as if the requested GiftCard doesn't exist — which should trigger a 404 Not Found response.

```
when(giftCardDao.findFilteredList(eq(1L),any(Map.class),eq(false))).thenReturn(Arrays.asList(card2));
```

When the `filterList` function is called for our artificial DAO with parameters(1L, any type of `hashMap`, false), trust me and just return `card2`.

```
@Test
@DisplayName("Verify getGiftCardByCommunityId functionality")
public void testGetGiftCardByCommunityId() {
    List<GiftCard> giftCards = new ArrayList<>();

    GiftCard card1 = new GiftCard();
    card1.setCommunityId(1L);
    card1.setCardIdentifier("APPLE");
    card1.setId(10L);
    giftCards.add(card1);

    GiftCard card2 = new GiftCard();
    card2.setCommunityId(1L);
    card2.setCardIdentifier("MANGO");
    card2.setId(40L);
    giftCards.add(card2);
}
```

```

GiftCard card3 = new GiftCard();
card3.setCommunityId(1L);
card3.setCardIdentifier("MANGO12");
card3.setId(50L);
giftCards.add(card3);

GiftCard card4 = new GiftCard();
card4.setCommunityId(1L);
card4.setCardIdentifier("MANGO12345");
card4.setId(60L);
giftCards.add(card4);

// Case 1: No filters - return all cards
when(giftCardDao.findGiftCardsByCommunity(eq(1L), isNull(),
isNull()))).thenReturn(giftCards);
    List<GiftCard> result1 = giftCardService.getAllGiftCardsForCommunity(1L,
null, null);
    assertEquals(4, result1.size());

// Case 2: Filter by exact cardIdentifier "MANGO12345" - return only card4
when(giftCardDao.findGiftCardsByCommunity(eq(1L), eq("MANGO12345"),
isNull()))
    .thenReturn(List.of(card4));
    List<GiftCard> result2 = giftCardService.getAllGiftCardsForCommunity(1L,
null, "MANGO12345");
    assertEquals(1, result2.size());
    assertEquals(60L, result2.get(0).getId());

// Case 3: Filter by exact cardIdentifier "MANGO" - should return only
card2
when(giftCardDao.findGiftCardsByCommunity(eq(1L), eq("MANGO"), isNull()))
    .thenReturn(List.of(card2));
    List<GiftCard> result3 = giftCardService.getAllGiftCardsForCommunity(1L,
null, "MANGO");
    assertEquals(1, result3.size());
    assertEquals(40L, result3.get(0).getId());

// Case 4: Filter by cardIdentifier "MANGO" and customerID 50L - simulate
returning card3 only

```



```

        when(giftCardDao.findGiftCardsByCommunity(eq(1L), eq("MANGO"), eq(50L)))
            .thenReturn(List.of(card3));
        List<GiftCard> result4 = giftCardService.getAllGiftCardsForCommunity(1L,
50L, "MANGO");
        assertEquals(1, result4.size());
        assertEquals(50L, result4.get(0).getId());
    }

```

- **You're mocking** `giftCardService.getAllGiftCardsForCommunity(...)` to return a list containing 2 gift cards — one with ID 1L (probably `giftCard`) and another with ID 20L (from `giftCardMock(20L)`).
- **This mock ensures** that no matter what input parameters are passed to the controller, the service returns that same list — this is because you're using `Mockito.anyLong()` and `Mockito.anyString()` in the first mock call.
- You're calling the real **controller endpoint** `/gift-cards/community` using `withJwtAuth(get(...))` and asserting:
 - HTTP 200 OK status ✓
 - HATEOAS `_links.self` exists ✓
 - Embedded list of gift cards has **the same size as the mocked list (2)** ✓
- In the **second request**, you're also testing the same endpoint **but without any query parameters**, which means you're now testing optional behavior as parameters are optional

👉 **“Regardless of what parameters the controller receives, always return this list I provide.”**

Because you mocked the `giftCardService`, and said `anyLong()` and `anyString()` it bypasses actual logic and database.

So what's really being tested here is:

- The **controller correctly handles a non-empty response** from the service
- It returns the right HTTP status (200)
- It embeds the list in the correct HATEOAS format

GiftCardServiceTest

`@ExtendWith(SpringExtension.class)`

Integrates Mockito with JUnit 5.

Allows annotations like `@Mock` and `@InjectMocks` to work properly.

GiftCardResource

```
/**
 * Constructor to create relational links * * @param giftCard GiftCard to
create resource for
 * @param links additional links to include
 */public GiftCardResource(GiftCard giftCard, Link[] links) {
    super(giftCard, links);
    final Long giftCardId = giftCard.getId();
    final Long customerId = giftCard.getResident() != null ?
giftCard.getResident().getId() : null;

    add(linkTo(methodOn(GiftCardController.class).getGiftCard(giftCardId)).withSelfRel());

    if (customerId != null) {

        add(linkTo(methodOn(CustomerController.class).getCustomer(customerId)).withRel("customer"));
    }
}
```

SelfRel means link to the giftcard Object itself. withRel("customer") means a link to customerObject.

What is `@Mock` doing? (Mockito)

`@Mock`

```
private GiftCardDao giftCardDao;
```

You're creating a mock of GiftCardDao:

You're not hitting the real database.

You control its output using `Mockito.when(...).thenReturn(...)`.

✅ Why mock it?

You're testing `GiftCardService`, not the database.

Mocking avoids flaky or slow tests caused by real DB behavior.

You have full control over DAO behavior in tests.

```
@Test
@DisplayName("Verify getGiftCardByCommunityId functionality")
public void testGetGiftCardByCommunityId() {
    List<GiftCard> giftCards = new ArrayList<>();
    for(int i = 0; i < 3; i++){
        GiftCard mock = giftCardMock(i+1L);
        mock.setCardIdentifier("ABCDE");
        giftCards.add(mock);
    }
    Map<String, Object> expectedMap = new HashMap<>();
    expectedMap.put("cardIdentifier", "ABCDE");
    expectedMap.put("customerId", 2L);

    when(giftCardDao.findFilteredList(1L, expectedMap,
false)).thenReturn(giftCards);

    List<GiftCard> result = giftCardService.getAllGiftCardsForCommunity(1L,
2L, "ABCDE");

    assertNotNull(result);
    assertEquals(3, result.size());
}
```

✅ `List<GiftCard> giftCards = new ArrayList<>();`

- This is a list that will hold **3 fake/mock gift cards**.
 - We're using this list to **simulate what the database would return** when queried.
-

✓ `for(int i = 0; i < 3; i++) { ... }`

- We're creating **3 mock gift cards**, with IDs `1`, `2`, and `3`.
- Inside the loop:
 - `giftCardMock(i+1L)` gives you a gift card with ID `i+1`
 - `mock.setCardIdentifier("ABCDE")` ensures all cards have the same `cardIdentifier`
 - You add each card to the `giftCards` list

→ So now `giftCards` looks like this: [
GiftCard{id=1, cardIdentifier="ABCDE"},
GiftCard{id=2, cardIdentifier="ABCDE"},
GiftCard{id=3, cardIdentifier="ABCDE"}
]

✓ `Map<String, Object> expectedMap = new HashMap<>();`

- This is a **filter map** that mimics how the service layer is supposed to call the DAO (database).
- We're saying: *"Only return cards that belong to customerId = 2 and cardIdentifier = ABCDE."*

→ `expectedMap` now contains: {
"cardIdentifier": "ABCDE",
"customerId": 2L
}



`when(giftCardDao.findFilteredList(...)).thenReturn(giftCards);`

- This is the **mocking part**. We're saying:

"If the DAO (`giftCardDao`) is called with `communityId = 1`, `filter map = expectedMap`, and `active = false`, then return the 3 gift cards I made earlier."

if `orderService.updateCheckOrderItem(...)` is called with any `dto`, `anyString`, `anyLong` values, just return `checkOrderItemMock`.

`when(orderService.updateCheckOrderItem(any(CheckOrderItemResource.CheckOrderItemDTO.class), anyString(), anyLong()))thenReturn(checkOrderItemMock);`

#UpdateCheckOrderItemControllerTest

```
@Test
@DisplayName("Test Update Check Order Item")
public void testUpdateCheckOrderItem() throws Exception {
    //create dto
    Map<String, Object> body = new HashMap<>();
    body.put("checkId", "1");
    body.put("orderId", "2");
    body.put("amount", "200");
    body.put("billLater", "Y");

    //success

    when(orderService.updateCheckOrderItem(any(CheckOrderItemResource.CheckOrderItemDTO.class), anyString(), anyLong()))
        .thenReturn(checkOrderItemMock);
    withJwtAuth(patch("/check-order-items/{checkOrderId}", 1L)
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(body)),
        userToken)
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.links").exists());

    //null parameter(no dto passed), should fail
    withJwtAuth(patch("/check-order-items/{checkOrderId}", 1L)
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(null)),
        userToken)
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.errors").exists());
}
```

We are creating a dto with certain parameter(checkID, orderItemID, amount, billLater) and testing for success

We are also testing if we passed in null instead of dto(meaning no dto passed), it should throw error.

###UpdateCheckOrderServiceTest

```
@Test
@DisplayName("Update Check Order Item")
public void testUpdateCheckOrderItem() {
    CheckOrderItemResource.CheckOrderItemDTO dto = new
    CheckOrderItemResource.CheckOrderItemDTO();

    dto.setAmount(BigDecimal.TEN);
    dto.setBillLater("N");
    dto.setCheckId(1L);
    dto.setOrderItemId(2L);

    // Create a CheckOrderItem object
    CheckOrderItem checkOrderItem = new CheckOrderItem();

    when(checkDao.findById(1L)).thenReturn(checkMock);
    when(orderItemDao.findById(2L)).thenReturn(orderItemMock);
    when(checkOrderItemDao.findById(123L)).thenReturn(checkOrderItem);

    CheckOrderItem result = orderService.updateCheckOrderItem(dto,
    "testUser1", 123L);

    //(TEST1: check if values are actually set and are called once)
    assertEquals(BigDecimal.TEN, result.getAmount());
    assertEquals("N", result.getBillLater());
    assertEquals(checkMock, result.getCheck());
    assertEquals(orderItemMock, result.getOrderItem());

    //Check if saveOrUpdate is called once(when calling updateCheckOrderItem)
    verify(checkOrderItemDao ,
    times(1)).saveOrUpdate(any(CheckOrderItem.class), eq("testUser1"));

    //(TEST2: Now check for other parameters)
    dto.setNumerator(BigDecimal.ZERO);
    dto.setDenominator(BigDecimal.TEN);
    dto.setAmount(BigDecimal.ONE);
```

```
result = orderService.updateCheckOrderItem(dto, "testUser1", 123L);

//check if values are actually set
assertEquals(BigDecimal.ONE, result.getAmount());
assertEquals("N", result.getBillLater());
assertEquals(checkMock, result.getCheck());
assertEquals(orderItemMock, result.getOrderItem());
assertEquals(BigDecimal.ZERO, result.getNumerator());
assertEquals(BigDecimal.TEN, result.getDenominator());

//Check if saveOrUpdate is called once(when calling updateCheckOrderItem)
verify(checkOrderItemDao ,
times(2)).saveOrUpdate(any(CheckOrderItem.class), eq("testUser1"));
}
```

✅ **List<GiftCard> result =**
giftCardService.getAllGiftCardsForCommunity(1L, 2L,
"ABCDE");

- This is the **actual method we're testing**.
- We're calling our `giftCardService`, passing in:
 - `communityId = 1`
 - `customerId = 2`
 - `cardIdentifier = "ABCDE"`

➡ What happens inside the method (in the service class):

1. It builds a map like `expectedMap`
2. It calls `giftCardDao.findFilteredList(1L, map, false)`
3. Because of our mock setup, it returns the `giftCards` list

✓ So `result` now holds the 3 mock gift cards we set up earlier.

🔧 **What is `@InjectMocks` doing?**

@InjectMocks

```
private GiftCardService giftCardService;
```

This tells Mockito:

“Create a real GiftCardService, and automatically inject any mocks (like giftCardDao) into it.”

✓ Why use @InjectMocks?

You want to test actual business logic in GiftCardService, but with mocked dependencies like the DAO.

Avoids manual setup or wiring.

The Test Method

@Test

@DisplayName("Verify getGiftCardByID functionality")

```
public void testGetGiftCardByID() {
```

```
    when(giftCardDao.findById(Mockito.anyLong())).thenReturn(giftCardMock);
```

```
        GiftCard result = giftCardService.getGiftCard(giftCardMock.getId());
```

```
        assertEquals(giftCardMock.getId(), result.getId());
```

```
}
```

🧠 What's happening:

You mock giftCardDao.findById(...) to return giftCardMock.

You call the real service method:

```
giftCardService.getGiftCard(id)
```

It returns the mocked GiftCard.

General Mockito Department Service Test

```
@Test
@DisplayName("Verify get departments by community only (no posMode filter)")
public void verifyGetDepartmentsByCommunity() {
    Department department1 = departmentMock(1L, 10L);
    Department department2 = departmentMock(2L, 10L);
    List<Department> departments = Arrays.asList(department1, department2);

    //When method is called with communityId = 10 return departments
    when(departmentDao.findByCommunityId(10L)).thenReturn(departments);

    List<Department> result =
departmentService.getDepartmentsForCommunity(10L, null);

    assertNotNull(result);
    assertEquals(2, result.size());
    verify(departmentDao, times(1)).findByCommunityId(eq(10L));
}
```

```
@Test
@DisplayName("Verify get departments by community with posMode filter")
public void verifyGetDepartmentsByCommunityWithPosModeFilter() {
    Department department1 = departmentMock(1L, 10L);
    department1.setPosMode("S");

    Department department2 = departmentMock(2L, 10L);
    department2.setPosMode("S");

    List<Department> departments = Arrays.asList(department1, department2);

    //When method is called with communityId = 10 and posMode = "S", return
departments
    when(departmentModeDao.findDepartmentsByPosMode(eq(10L),
eq("S"))).thenReturn(departments);

    List<Department> result =
departmentService.getDepartmentsForCommunity(10L, "S");

    assertNotNull(result);
```

```

    assertEquals(2, result.size());
    assertEquals("S", result.get(0).getPosMode());

    verify(departmentModeDao, times(1)).findDepartmentsByPosMode(eq(10L),
eq("S"));
}

```

You assert that the returned card's ID matches the mock — verifying correct delegation.

Summary of Why/What

Annotation/Code	Purpose
@Mock	Creates a fake <code>GiftCardDao</code> to avoid real DB calls
@InjectMocks	Builds a real <code>GiftCardService</code> and injects the mocked DAO
when(...).thenReturn(...)	Controls what the mock DAO returns during test
assertEquals(...)	Validates that the service behaves as expected

Why Mocking Matters

Without Mocking	With Mocking
Calls real database (slow, flaky)	Test is fast, predictable, and isolated
Hard to simulate error/failure cases	Can simulate any behavior with <code>.thenReturn()</code>
Can't test service logic in isolation	Only the service logic is tested

Criteria-Based Api (Hibernate)

https://www.tutorialspoint.com/hibernate/hibernate_criteria_queries.htm

```

/**
 * Retrieves a list of unique {@link Department} entities for the given

```

```

community
    * where either the department's {@code posMode} or the associated mode's
    {@code posMode}
    * matches the provided value. *This query runs on the {@link DepartmentMode}
mapping table and extracts all departments
    * that satisfy the following: *      {@code communityId} matches
    *      {@code department.posMode == posMode} OR {@code mode.posMode ==
posMode}
    * * @param communityId the ID of the community to filter departments by
    * @param posMode the POS mode string to match against department or mode
    * @return a list of matching {@link Department} objects, or {@code null} if
none found
    */
public List<Department> findDepartmentsByPosMode(Long communityId, String
posMode) {
    Criteria criteria = getBaseCriteria();

    //Filter by communityId
    criteria.add(Restrictions.eq("communityId", communityId));

    //Filter by posMode matching either department.posMode OR mode.posMode
    Criterion departmentPosMode = Restrictions.eq("department.posMode",
posMode);
    Criterion modePosMode = Restrictions.eq("mode.posMode", posMode);

    LogicalExpression orExp = Restrictions.or(departmentPosMode,
modePosMode);
    criteria.add(orExp);

    // Return DepartmentMode entities
    List<DepartmentMode> departmentModes = (List<DepartmentMode>)
criteria.list();

    // Extract unique departments from the results
    List<Department> departments = departmentModes.stream()
        .map(DepartmentMode::getDepartment)
        .distinct()
        .collect(Collectors.toList());

```

```
return departments.isEmpty() ? null : departments;  
}
```